

Inheritance in java

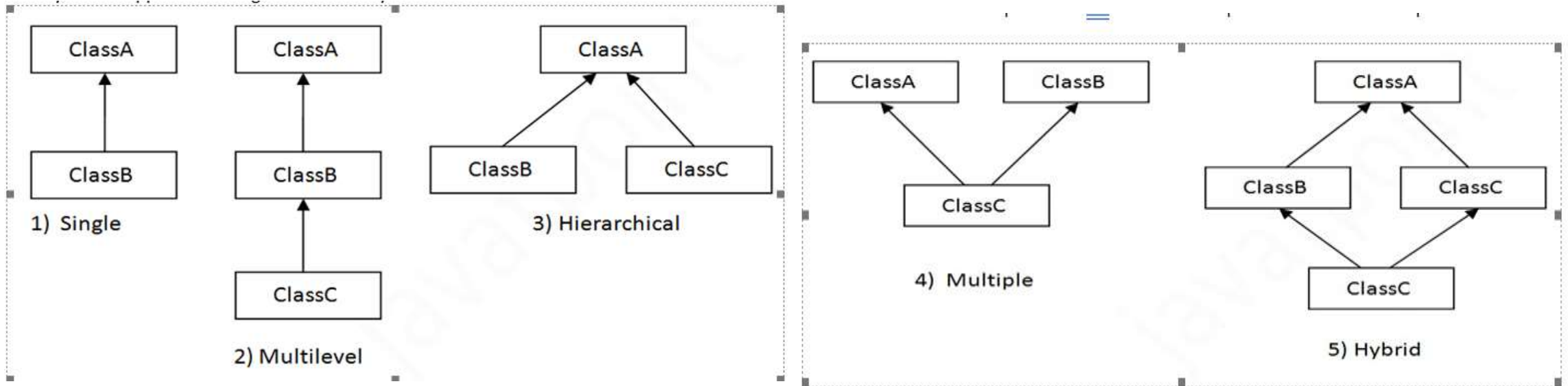
- It means transferring properties from one class to another in java , properties of one class can be transfer to another class is known as inheritance.
- Extends keywords= is used to perform inheritance in java.

Types of inheritance

- 1.Single Inheritance
- 2.Multilevel Inheritance
- 3.Multiple Inheritance
- 4.Hierarchical
- 5. Hybrid Inheritance

Types of inheritance

- On the basis of class, there can be three types of inheritance: single, multilevel and hierarchical. Multiple and Hybrid is supported through interface only. We will learn about interfaces later.



Syntax of Inheritance

```
class Subclass-name extends Superclass-name
{
    //methods and fields
}
```

Single Inheritance

```
class A
{
    void msg()
    {
        System.out.println("I Am Class A");
    }
}

class B extends A
{
    void disp()
    {
        System.out.println("I Am Class B");
    }
}
```

```
class Check
{
    public static void main(String args[])
    {
        B b1=new B();

        b1.msg();

        b1.disp();
    }
}
```

Note-: When a class inherits another class , it is known as single inheritance .In single inheritance only one super class and one is sub class.

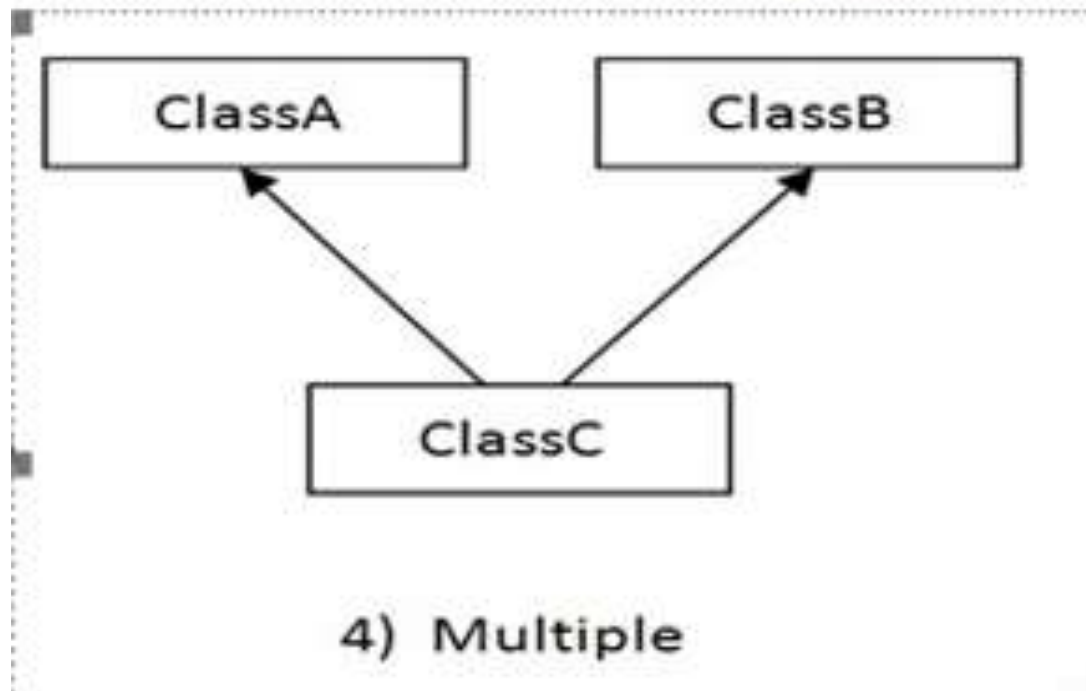
Multilevel Inheritance

```
class A
{
    void msg()
    {
        System.out.println("I Am Class A");
    }
}
class B extends A
{
    void disp()
    {
        System.out.println("I Am Class B");
    }
}
class C extends B
{
    void show()
    {
        System.out.println("I Am Class C");
    }
}
```

```
class Check
{
    public static void main(String args[])
    {
        C c1=new C();
        c1.msg();
        c1
    }
}
```

Multiple Inheritance

- Whenever two or class transfer it's property into single class is known as multiple inheritance.
- Java does not support it because of ambiguity(duplication) problem.



Hierarchical

- Whenever single class transfer its property into more than one class is known as Hierarchical inheritance.

```
class BTech
{
    void show()
    {
        System.out.println("method of B.Tech class");
    }
}
class MCA extends BTech
{
    void disp()
    {
        System.out.println("method of MCA class");
    }
}
class MBA extends BTech
{
    void disp2()
    {
        System.out.println("method of MBA class");
    }
}
```

Hierarchical

```
class Check
{
    public static void main(String args[])
    {
        MBA m1=new MBA();
        m1.show();
        m1ss.disp();
        m1.disp2();
    }
}
```

Method Overriding

- Whenever base class(super class) and derived class both have same name method than base class method lost it's definition is known as method overriding.

```
class A
{
    void show()
    {
        System.out.println("method of A class");
    }
}
class B extends A
{
    void show()
    {
        System.out.println("method of B class");
    }
}
class Check
{
    public static void main(String args[])
    {
        B b1=new B();
        b1.show();
    }
}
```

Output-:
method of B class

Super keyword

- To overcome the problem of method overriding we can use 'super' keyword.
- It is used to call base class method when base class and derived class have same name method .

```
class A
{
    void show()
    {
        System.out.println("method of A class");
    }
}
class B extends A
{
    void show()
    {
        super.show();
        System.out.println("method of B class");
    }
}
class Check
{
    public static void main(String args[])
    {
        B b1=new B();
        b1.show();
    }
}
```

Output-:
method of A class
method of B class

final keyword

- Final is keyword that can be used with the variable, method and class.

final with variable(constant)

final with method(Restrict overriding)

final with class(Restrict inheritance)

Final variable-: It creates constant value that means it never change their value during execution.

class F1

{

final float f=3.14f;

public static void main(String args[])

{

F1 c=new F1();

c.f=3.14f+2.14f;

System.out.println(c.f);

}

}

Error-: cannot assign a value to final variable f

final with method

Final keyword is used with the method which restrict overriding that means derived class can not create same name method and could not change their signature.

```
class A
{
    final void show()
    {
        System.out.println("method of base class");
    }
}
class B extends A
{
    void show()
    {
        System.out.println("Method of derived class");
    }
}
class F1{
    public static void main(String args[]){
        B b1=new B();
        b1.show();
    }
}
```

Error:-code produce an error that is show() can not override because created as final

Final with class

- It can be used with the class to restrict inheritance that means user can create instance of that class but could not inherit it.

```
final class A{
    void show1()
    {
        System.out.println("method of base class");
    }
}

class B extends A
{
    void show2(){
        System.out.println("Method of derived class");
    }
}

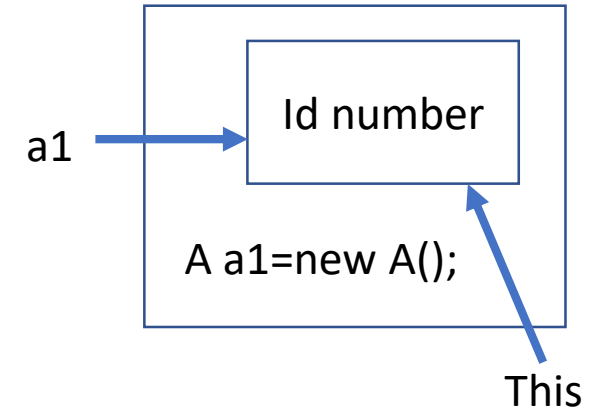
class F1{
    public static void main(String args[]){
        B b1=new B();
        b1.show1();
        b1.show2();
    }
}
```

this keyword

- 1. this keyword refers to the current object inside a method or constructor.

Ex:

```
class A
{
    void show()
    {
        System.out.println(this);
    }
}
public static void main(String args[])
{
    A a1=new A();
    System.out.println(a1);
    a1.show();
}
```



Output:-
A@36baf30c
A@36baf30c

this keyword

- Whenever the name of instance and local variable both are same then our runtime environment JVM gets confused that which one is local variable and which one is instance variable, to avoid this problem we should use this keyword.

```
class A{
    int a;
    A(int a)
    {
        this.a=a;
    }
    void show(){
        System.out.println(a);
    }
    public static void main(String args[]){
        A a1=new A(200);
        a1.show();
    }
}
```



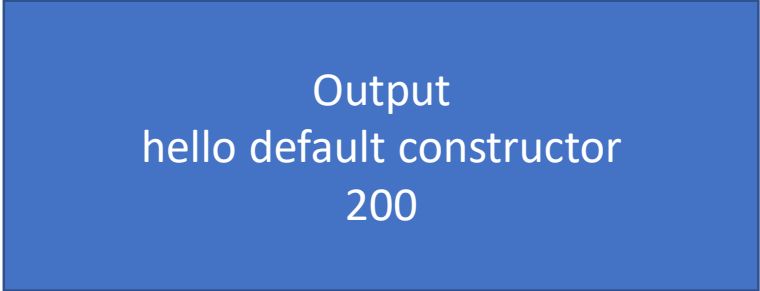
Ensure a is instance variable(this refer to current object, here instance variable a is part of object).

Output
200

this keyword

3. It is also used when we want to call the default constructor of it's own class.

```
class A{
    A()
    {
        System.out.println("hello default constructor");
    }
    A(int a)
    {
        this();//called default constructor
        System.out.println(a);
    }
    public static void main(String args[])
    {
        A a1=new A(200);
    }
}
```

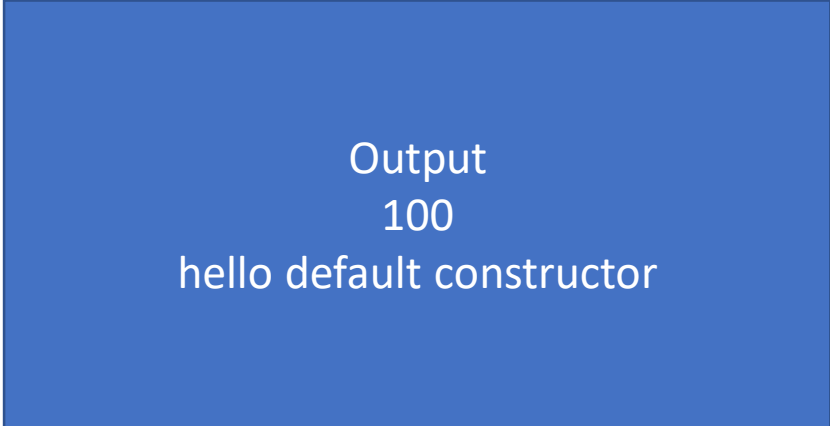


Output
hello default constructor
200

this keyword

- It is also called parametrized constructor of its own class.

```
class A
{
    A()
    {
        this(100); //called default constructor
        System.out.println("hello default constrcutor");
    }
    A(int a)
    {
        System.out.println(a);
    }
    public static void main(String args[])
    {
        A a1=new A();
    }
}
```



Output
100
hello default constructor

super keyword

- Super keyword refer to the object of super class, it is used when we want to call the super class variable, method, and constructor through sub class object
- Note:-Whenever the super class & sub class variable and method name both are same than it can be used only.

Example with variable

Class A

```
{
    int a=10;
}class B extends A
{
    int a=90;
    void show()
    {
        System.out.println(a);
    }
}
```

Class Check

```
{
    public static void main(String args[])
    {
        B b1=new B();
        b1.show();
    }
}
```